Lecture 1~6 보충: 직관적 이해를 위한 심화 해설

> 강의 자료를 읽으면서 "왜?", "어떻게 이 아이디어가 나왔지?"라는 질문이 드는 부분들을 모아 직관적으로 설명합니다. 필요한 곳에 Mermaid 다이어그램을 포함합니다.

Lecture 1: VAE

1.1 왜 "확률 분포"를 학습하는가? — 생성 모델의 근본 동기

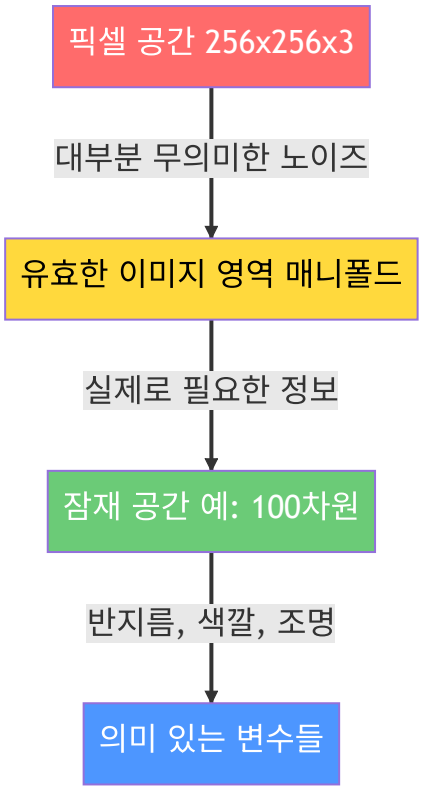
"데이터의 확률 분포 $p(x)$를 학습한다"는 말이 추상적으로 느껴질 수 있다. 핵심은 이것이다:

주사위의 비유: 공정한 주사위의 확률 분포를 알면 ($p(1) = p(2) = \cdots = p(6) = 1/6$), 그 분포에서 샘플링하여 "주사위를 굴린 것 같은" 결과를 무한히 만들어낼 수 있다. 마찬가지로, "사람 얼굴 이미지의 확률 분포"를 알면 거기서 샘플링하여 새로운 얼굴을 무한히 만들어낼 수 있다.

그런데 이미지 분포는 주사위처럼 간단하지 않다. 256×256×3 이미지라면 가능한 조합이 256^{196608}개다. 이 중 "사람 얼굴처럼 보이는" 이미지는 극히 일부이고, 나머지는 무의미한 노이즈다. 생성 모델은 이 거대한 공간에서 "의미 있는 이미지가 몰려 있는 영역"의 확률이 높도록 분포를 학습하는 것이다.

1.2 Manifold가 왜 중요한가? — 차원의 저주와의 싸움

강의에서 매니폴드 개념이 등장하는데, 이것이 왜 핵심인지 더 깊이 이해하자.



비유: 지구 표면은 3차원 공간에 있지만, 표면 자체는 2차원이다 (위도, 경도만 있으면 위치를 특정할 수 있다). 마찬가지로, 얼굴 이미지는 196,608차원 공간에 있지만, 실제로 변하는 것은 "나이, 성별, 표정, 조명" 등 수십~수백 차원의 변수뿐이다.

오토인코더와 VAE는 이 매니폴드를 찾는 도구다. 인코더가 고차원→저차원으로 압축하는 것은 "매니폴드 위의 좌표를 찾는 것"이고, 디코더가 저차원→고차원으로 복원하는 것은 "좌표에서 원래 공간의 점을 복원하는 것"이다.

1.3 왜 Prior를 표준 정규분포로 설정하는가?

$p(z) = \mathcal{N}(0, I)$로 가정하는 것이 자의적으로 보일 수 있다. 왜 하필 표준 정규분포인가?

이유 1: 샘플링의 용이성 생성 시 z를 뽑아야 하는데, 표준 정규분포는 샘플링이 극도로 쉽다 (`torch.randn()`). 복잡한 분포에서 샘플링하는 것은 그 자체로 어려운 문제다.

이유 2: 디코더의 표현력에 의존 VAE 논문 저자 Kingma의 핵심 통찰: "prior가 단순해도, 디코더의 깊은 뉴럴넷이 복잡한 매핑을 학습한다." 즉, z의 분포가 단순해도 디코더가 충분히 강력하면 복잡한 이미지를 만들어낼 수 있다.

비유: 피아노 건반(prior)은 단순한 1차원 배열이지만, 건반 위에 올려진 손(디코더)이 복잡한 멜로디를 만들어낸다. 건반 자체가 복잡할 필요는 없다.

이유 3: KL divergence의 closed-form 두 가우시안 사이의 KL divergence는 해석적 공식이 있어 계산이 매우 효율적이다. prior가 복잡한 분포이면 KL divergence를 닫힌 형

태로 계산할 수 없다.

1.4 KL Divergence — 직관적으로 무엇을 재는가?

$$D_{KL}(q\|p) = \mathbb{E}_q\left[\log\frac{q(x)}{p(x)}\right]$$

이 수식이 "두 분포의 차이"를 잰다고 하는데, 정확히 어떤 의미인가?

정보이론적 해석: q를 사용해서 데이터를 압축했을 때의 평균 비트 수와, p를 사용해서 압축했을 때의 평균 비트 수의 **차이**다.

일상 비유: 서울의 날씨 분포가 p (맑음 70%, 비 30%)인데, 당신이 q (맑음 50%, 비 50%)라고 잘못 믿고 있다면? 매일 우산을 챙길지 결정할 때 **평균적으로 손해를 보는 정도**가 KL divergence다. $q = p$이면 손해가 0이고, 차이가 클수록 손해가 크다.

왜 비대칭인가? $D_{KL}(q\|p) \neq D_{KL}(p\|q)$인 이유:

- $D_{KL}(q\|p)$: "q를 기준으로 p를 평가" → q가 높은데 p가 낮은 곳에 큰 벌점
- $D_{KL}(p\|q)$: "p를 기준으로 q를 평가" → p가 높은데 q가 낮은 곳에 큰 벌점

VAE에서 $D_{KL}(q_\phi(z|x)\|p(z))$를 쓰는 이유: 인코더 분포 q가 prior p보다 넓은 것은 허용하되, q가 집중된 곳에서 p가 비어있으면 안 된다는 의미. 즉 "인코더가 만든 분포는 prior가 커버하는 범위 안에 있어야 한다."

1.5 ELBO 유도에서 "1을 곱하는 트릭" — 왜 이런 게 통하는가?

ELBO 유도의 핵심 트릭은 $\int q_\phi(z|x)dz = 1$을 곱하는 것이다. 이것이 왜 강력한가?

핵심: 우리는 $\log p_\theta(x)$를 최적화하고 싶은데, 여기에 인코더 q_ϕ가 없다. 하지만 인코더를 도입해야 효율적인 학습이 가능하다. "1을 곱하기"는 **식을 변형하지 않으면서 원하는 항을 등장시키는 트릭**이다.

수학에서 자주 쓰는 패턴이다:

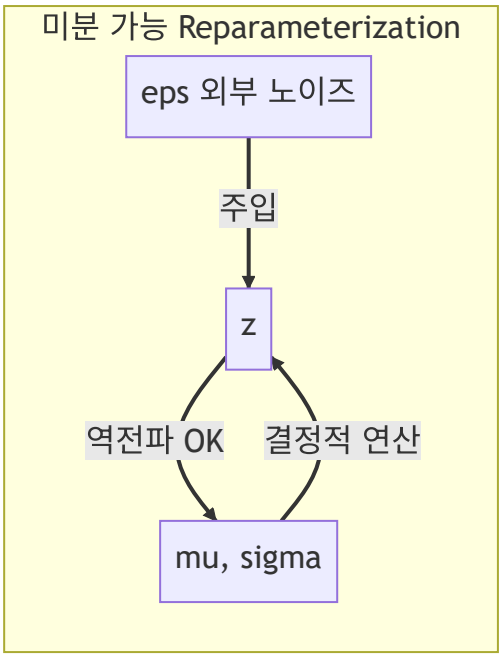
- $a = a \cdot 1 = a \cdot \frac{b}{b}$ → $\frac{b}{b}$를 곱해서 b를 등장시킴
- $\log p(x) = \log p(x) \cdot \int q(z|x)dz$ → $q(z|x)$를 등장시킴

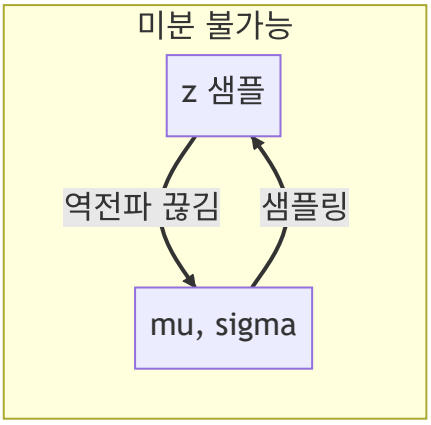
이후 $\log p(x)$가 z에 무관한 상수라서 적분 안으로 들어갈 수 있고, Bayes' rule과 로그 분리를 거쳐 ELBO가 튀어나온다. 이 과정 자체는 항등 변환(등식이 계속 유지됨)이므로 어떤 근사도 없다. 근사가 들어오는 유일한 지점은 마지막에 $D_{KL}(q\|p_\theta(z|x)) \geq 0$을 이용해 부등식으로 바꿀 때다.

1.6 Reparameterization Trick — 왜 "샘플링"이 미분 불가능한가?

이것은 많은 사람이 헷갈리는 부분이다.

문제의 본질: 역전파는 "출력의 변화량이 입력의 변화량에 어떻게 의존하는가"를 추적한다. $z = \text{sample}(\mu, \sigma)$라고 하면, μ를 조금 바꿨을 때 z가 어떻게 바뀌는지를 알아야 한다. 그런데 "샘플링"은 확률적 과정이라 μ와 z 사이에 결정적 함수 관계가 없다.





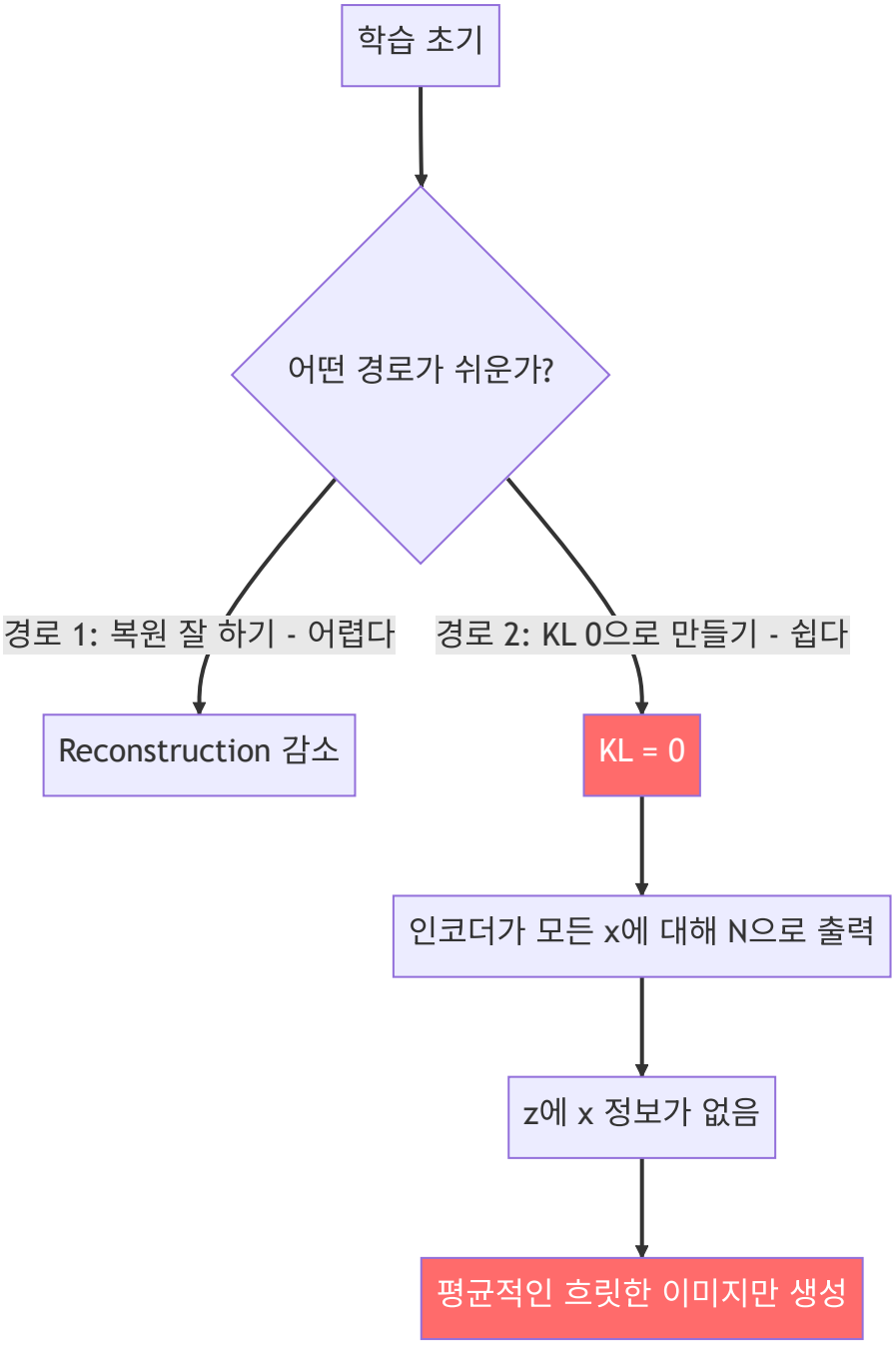
비유:

- **before**: "이 주사위를 굴려서 나온 값" → 주사위 제조사(μ, σ)를 바꿔도 결과가 어떻게 바뀔지 모름
- **after**: "표준 주사위를 굴린 값(ε)에 가중치(σ)를 곱하고 편향(μ)을 더한 값" → μ를 1 늘리면 z도 정확히 1 늘어남

핵심: 확률적 요소(ε)를 외부로 빼내서, μ와 σ에 대한 결정적 함수로 만든 것이다.

1.7 Posterior Collapse — 왜 발생하고 왜 문제인가?

근본 원인: VAE의 손실 함수는 Reconstruction + KL 두 항의 합이다. 학습 초기에 디코더가 아직 약할 때, KL 항을 0으로 만드는 것이 **가장 쉬운 손실 감소 경로**다.



비유: 학생(모델)에게 시험(loss) 점수를 높이라고 했더니, 공부(reconstruction)는 안 하고 출석점수(KL=0)만 채워서 최소 학점을 맞춘 것이다.

해결 방법들:

- **KL annealing**: 학습 초기에 KL 항의 가중치를 0에서 시작해 서서히 올림
- **Free bits**: KL이 특정 임계치 이하면 벌점을 주지 않음
- **VQ-VAE**: 이산 코드북을 강제해서 정보 병목을 물리적으로 생성

Lecture 2: PixelRNN/CNN, VQ-VAE, Normalizing Flow

2.1 자기회귀(Autoregressive) 모델 — 왜 "곱"으로 분해하는가?

$$p(x) = \prod_{i=1}^{n^2} p(x_i | x_1, \ldots, x_{i-1})$$

이것은 확률의 chain rule이라서 **항상 성립하는 등식**이다 (근사가 아니다). 그런데 왜 이렇게 분해하는 게 유용한가?

핵심 통찰: 전체 이미지의 결합 확률 $p(x_1, x_2, \ldots, x_{n^2})$을 한 번에 모델링하는 것은 불가능하다 (경우의 수가 256^{n^2}). 하지만 조건부 확률 $p(x_i | x_{<i})$는 **하나의 픽셀에 대한 256-way 분류 문제**로 환원되어, 뉴럴넷으로 모델링할 수 있다.

비유: 소설 전체를 한 번에 쓰는 것은 불가능하지만, "지금까지 쓴 문장을 보고 다음 단어 하나를 예측하는 것"은 가능하다. 이것을 반복하면 소설 전체가 완성된다. GPT가 정확히 이 원리로 동작한다. PixelRNN/CNN은 이미지 버전의 GPT인 셈이다.

2.2 LSTM — 왜 이미지에 RNN을 쓰는가?

이미지를 RNN으로 처리한다는 발상이 낯설 수 있다. 핵심을 짚으면:

LSTM 자체의 직관: 일반 RNN은 $h_t = \tanh(W[h_{t-1}, x_t])$인데, 이 행렬곱이 반복되면 기울기가 소멸/폭발한다.

LSTM의 핵심은 **Cell State**라는 "컨베이어 벨트"를 추가한 것이다:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

이것이 왜 기울기 소멸을 막는가? C_{t-1}에서 C_t로의 변환이 **덧셈**이기 때문이다. 역전파 시:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t$$

이것은 **행렬곱이 아니라 원소별 곱셈**이다. f_t가 1에 가까우면 기울기가 거의 그대로 전달된다. ResNet의 skip connection ($y = x + F(x)$)이 기울기 소멸을 막는 것과 정확히 같은 원리다.

이미지에서의 LSTM: 이미지를 "왼쪽 위에서 오른쪽 아래로 한 줄씩 읽는 텍스트"로 취급한다. 각 픽셀은 "단어"에 해당하고, LSTM이 이전 픽셀들의 정보를 기억하면서 다음 픽셀을 예측한다.

2.3 컨볼루션(Convolution) — 왜 도입하는가?

이것은 가장 근본적인 질문 중 하나다.

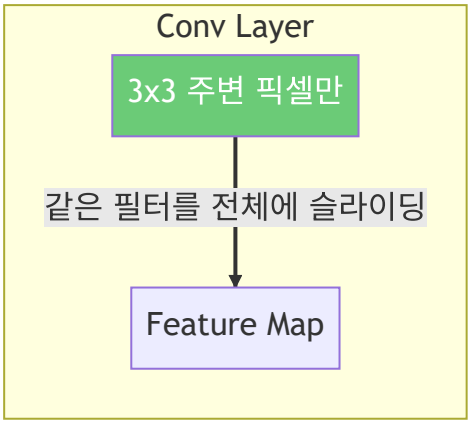
문제: Fully Connected로 이미지를 처리하면?

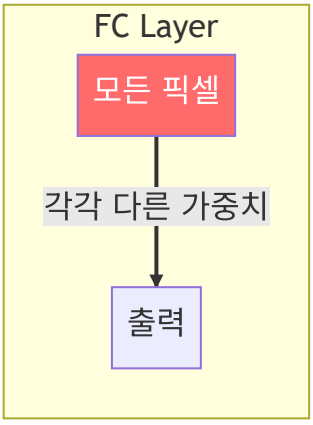
256×256 이미지를 FC 레이어에 넣으면 $65536 \times 65536 \approx 43$억 개의 파라미터가 필요하다. 불가능할 뿐 아니라, **이미지의 본질을 무시**하는 것이다.

이미지의 두 가지 핵심 성질:

성질	의미	FC의 문제
지역성 (Locality)	고양이 귀를 인식하려면 귀 주변만 보면 됨	모든 픽셀을 전부 연결 (낭비)
평행이동 불변성 (Translation Invariance)	고양이 귀가 왼쪽에 있든 오른쪽에 있든 "귀"임	위치마다 다른 가중치 (같은 패턴을 여러 번 학습)

컨볼루션은 이 두 성질을 구조적으로 반영한다:





- **지역성**: 3×3 필터는 주변 9개 픽셀만 본다 → 파라미터 9개
- **불변성**: 같은 필터를 이미지 전체에 공유 → 어디서든 같은 패턴을 감지

컨볼루션 연산이란?

$$\text{출력}(i,j) = \sum_m \sum_n \text{입력}(i+m, j+n) \cdot \text{필터}(m,n)$$

필터를 이미지 위에 올려놓고, 겹치는 부분의 **가중합**을 구하는 것이다. 필터가 [1, 0, -1]이면 "수평 엣지"를 감지하고, 대각선 패턴이면 "대각선 엣지"를 감지한다.

2.4 Masked Convolution — 왜 마스크가 필요한가?

PixelCNN에서 "미래 픽셀을 보면 안 된다"는 제약이 있다. 하지만 일반 컨볼루션은 중심 픽셀 주변 모든 방향을 본다.

해결: 필터의 아래쪽 절반과 오른쪽을 0으로 마스킹한다.

```
마스크 (5×5 필터):
1 1 1 1 1
1 1 1 1 1
1 1 ★ 0 0    ← ★ = 현재 예측 위치
0 0 0 0 0
0 0 0 0 0
```

이렇게 하면 컨볼루션을 적용해도 **이미 생성된 픽셀(위쪽, 왼쪽)만 참조**하게 된다. RNN처럼 순차적으로 처리하지 않고도, 자기회귀적 의존성을 보장할 수 있다.

Type A vs Type B:

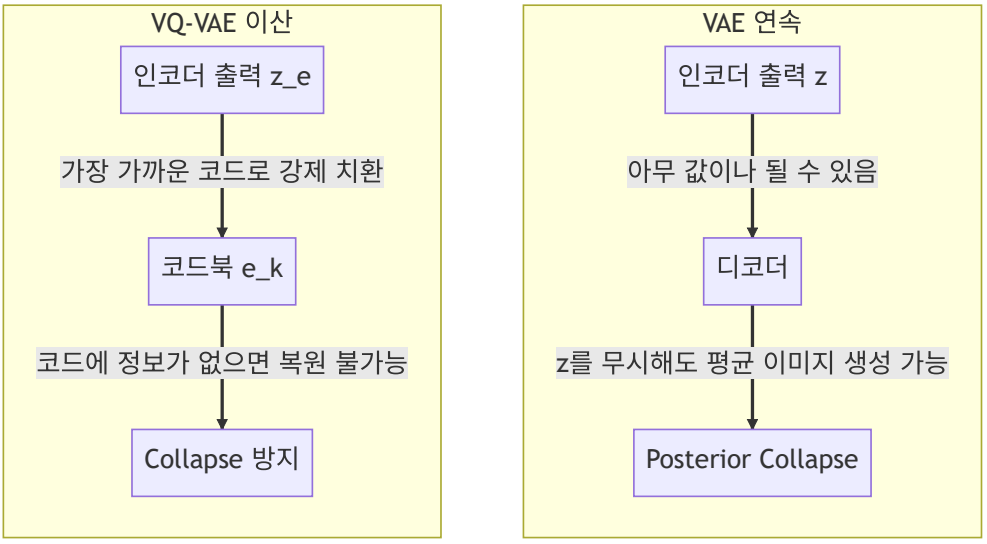
- **Type A** (첫 층): 중심 픽셀 자신도 제외 → "내 값을 모르는 상태에서 내 값을 예측"
- **Type B** (이후 층): 중심 포함 → 이전 층에서 이미 계산된 값이므로 사용 가능

2.5 코드북(Codebook) — 왜 연속 공간 대신 이산 공간을 쓰는가?

VQ-VAE의 핵심 아이디어를 더 깊이 파보자.

VAE의 근본적 문제: 연속 잠재 공간에서 posterior collapse가 발생한다. 디코더가 강해지면 z를 무시하고, 인코더는 $z = \mathcal{N}(0, I)$만 출력한다.

코드북이 이것을 해결하는 이유:



코드북은 **강제적인 정보 병목**을 만든다. 디코더는 코드북 벡터만 보고 이미지를 복원해야 하므로, 코드에 의미 있는 정보가 담기지 않으면 복원이 불가능하다. 이것은 VAE의 "부드러운" 정보 흐름과 대비된다.

왜 다른 이산화 방법이 아니라 코드북인가?

방법	문제
이진 코드 (0/1)	표현력 부족 — 64비트로 2^{64}가지밖에 표현 못함
K-means 클러스터링	엔드투엔드 학습 불가 — 인코더와 별개로 돌아감
Gumbel-Softmax	근사적이고 온도 파라미터 튜닝 필요
코드북 (VQ)	정확한 이산화 + 엔드투엔드 학습 가능 (STE로)

2.6 Straight-Through Estimator — argmin을 어떻게 역전파하는가?

VQ-VAE에서 가장 트릭키한 부분이다. `argmin`은 미분 불가능한 연산인데, 어떻게 학습하는가?

문제: 순전파에서 $z_q = e_k$ ($k = \text{argmin}_j \|z_e - e_j\|$) 이 과정은 이산적 선택이라 기울기가 정의되지 않는다.

해결 (Straight-Through Estimator): "순전파는 양자화된 값 z_q를 사용하되, 역전파는 z_e에 직접 기울기를 전달한다."

코드로 보면 매우 간단하다:

```
z_q = z_e + (z_q - z_e).detach()  # 순전파: z_q, 역전파: z_e
```

왜 이것이 합리적인가? z_e와 z_q가 충분히 가깝다면 (commitment loss가 이를 보장), z_q에 대한 기울기는 z_e에 대한 기울기와 거의 같다고 근사할 수 있다. 엄밀하지는 않지만, 실전에서 매우 잘 작동한다.

2.7 Change of Variables — Normalizing Flow의 핵심

Normalizing Flow를 이해하려면 "변수 변환 정리"를 직관적으로 이해해야 한다.

문제: $z \sim p(z)$이고 $x = f(z)$일 때, $p(x)$는 무엇인가?

1차원 직관: 고무줄을 생각하자. z 축에 균일하게 분포된 점들을 f라는 함수로 변환하면, 고무줄이 늘어나는 곳에서는 점들이 희박해지고, 수축하는 곳에서는 밀집된다.

$$p(x) = p(z) \cdot \left|\frac{dz}{dx}\right| = p(z) \cdot \left|\frac{1}{f'(z)}\right|$$

다차원으로 확장: 1차원에서 $|f'(z)|$가 "늘어남/수축"을 나타냈듯이, 다차원에서는 **자코비안 행렬의 행렬식(determinant)** $|\det J|$가 "부피 변화율"을 나타낸다.

$$p(x) = p(z) \cdot |\det J_{f^{-1}}(x)|$$

왜 Flow에서 행렬식 계산이 중요한가? 일반적으로 $n \times n$ 행렬의 행렬식은 $O(n^3)$이다. 잠재 공간 차원이 수만이면 계산 불가능. 그래서 Flow 모델은 **자코비안이 삼각 행렬이 되는** 특수한 변환만 사용한다. 삼각 행렬의 행렬식은 대각 원소의 곱이므로 $O(n)$이다.

2.8 Affine Coupling Layer — 왜 이렇게 설계하는가?

Normalizing Flow에서 Affine Coupling Layer가 등장하는데, 설계 의도를 이해하면 외울 필요가 없다.

요구사항: (1) 가역적이어야 함 (2) 자코비안 계산이 쉬워야 함

해결: 입력을 반으로 쪼갠다: $x = [x_1, x_2]$

$$y_1 = x_1$$

$$y_2 = x_2 \odot \exp(s(x_1)) + t(x_1)$$

왜 이것이 가역적인가? $y_1 = x_1$이므로 x_1은 즉시 복원 가능하고, x_1을 알면 $s(x_1)$과 $t(x_1)$을 계산할 수 있으므로 $x_2 = (y_2 - t(x_1)) \odot \exp(-s(x_1))$로 복원 가능하다.

왜 자코비안이 쉬운가? 자코비안이 하삼각 행렬이 되어, 행렬식 = 대각 원소의 곱 = $\prod \exp(s_i) = \exp(\sum s_i)$이다.

x_1이 변환되지 않는 문제: $y_1 = x_1$이므로 절반은 변하지 않는다. 이를 해결하기 위해 **교대 분할**을 사용한다 — 다음 레이어에서는 x_1과 x_2의 역할을 바꾼다.

Lecture 3: GAN

3.1 Minimax Game — 왜 이런 구조가 필요한가?

GAN 이전의 생성 모델(VAE, Flow)은 "확률 분포를 명시적으로 학습"했다. 그런데 왜 다른 접근이 필요했나?

VAE의 한계: ELBO를 최적화하므로 true likelihood의 하한만 다룬다. 그 결과 이미지가 흐릿하다 — "평균적으로 안전한" 이미지를 생성하는 경향이 있다.

GAN의 발상 전환: "분포를 직접 학습하지 말고, **진짜와 구분 불가능한 이미지를 만들면 되는 것 아닌가?**"

이것은 시험에 비유하면:

- **VAE 방식**: 교과서를 외워서 답안을 작성 (정석적이지만 밋밋함)
- **GAN 방식**: 모범 답안과 내 답안을 비교하며 개선 (채점관을 속일 수 있으면 합격)

3.2 BCE Loss와 GAN의 관계 — 왜 Binary Cross-Entropy인가?

GAN의 목적 함수:

$$V = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

이것이 실은 **Binary Cross-Entropy**와 같다는 것이 핵심이다.

BCE의 일반 형태: $-[y \log \hat{y} + (1-y)\log(1-\hat{y})]$

- 진짜 이미지 ($y = 1$): $-\log D(x)$ → D가 1을 출력하면 loss 0
- 가짜 이미지 ($y = 0$): $-\log(1 - D(G(z)))$ → D가 0을 출력하면 loss 0

즉, **판별자의 관점에서 GAN은 단순한 이진 분류 문제**다. 특별할 것이 없다. 특별한 것은 **분류 대상(가짜 이미지)이 학습 중에 계속 변한다**는 점이다.

3.3 Non-Saturating Loss — 왜 원래 손실로는 안 되는가?

이것은 수학적으로 매우 직관적이다.

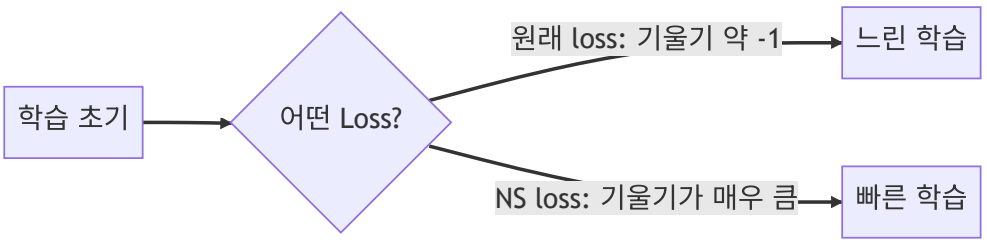
원래 생성자 손실: $\log(1 - D(G(z)))$

$D(G(z))$를 a로 놓으면:

- 학습 초기: 생성 이미지가 쓰레기 → D가 쉽게 구분 → $a \approx 0$
- $a \approx 0$에서 $\log(1-a)$의 기울기: $\frac{-1}{1-a} \approx -1$ (작은 기울기)
- **학습이 가장 필요한 시점에 기울기가 가장 작다!**

Non-Saturating Loss: $-\log D(G(z))$

- $a \approx 0$에서 $-\log(a)$의 기울기: $\frac{-1}{a} \to -\infty$ (매우 큰 기울기)
- **학습이 가장 필요한 시점에 기울기가 가장 크다!**



3.4 최적 판별자가 $\frac{1}{2}$가 되는 이유

$p_g = p_{\text{data}}$일 때 $D^*(x) = \frac{p_{\text{data}}}{p_{\text{data}}+p_g} = \frac{1}{2}$

직관: 생성 분포가 실제 분포와 완전히 같으면, 어떤 이미지를 봐도 "진짜일 확률 50%, 가짜일 확률 50%"다. 동전 던지기와 같다. 더 이상 구분할 수 있는 단서가 없으므로, 판별자는 찍을 수밖에 없다.

3.5 JS Divergence — 왜 KL이 아니라 JS인가?

GAN을 분석하면 최적 판별자 하에서 생성자는 **JS divergence**를 최소화한다:

$$JS(p\|q) = \frac{1}{2}D_{KL}(p\|m) + \frac{1}{2}D_{KL}(q\|m), \quad m = \frac{p+q}{2}$$

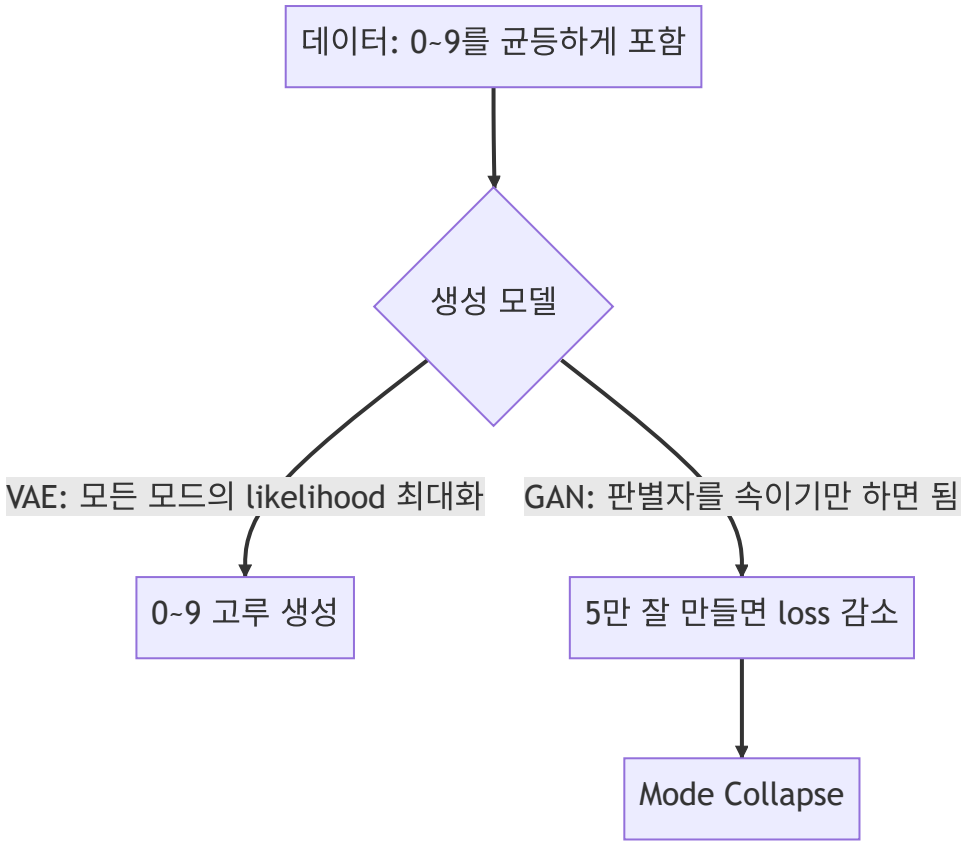
KL vs JS:

- **KL**: 비대칭. $D_{KL}(p\|q) \neq D_{KL}(q\|p)$. 한쪽이 0인데 다른 쪽이 양수이면 무한대.
- **JS**: 대칭. 항상 유한하고 $[0, \log 2]$ 범위. 더 안정적.

GAN의 minimax 구조가 자연스럽게 JS divergence를 최소화하는 것으로 귀결된다는 것은 이론적으로 우아한 결과다.

3.6 Mode Collapse — 왜 GAN에서만 특히 심각한가?

핵심 차이: VAE는 **모든 데이터의 likelihood를 올리는 것**이 목표다. 즉, 숫자 0~9 모두의 확률이 높아야 한다. 반면 GAN은 **판별자를 속이기만 하면 된다**. 숫자 5만 잘 만들어도 판별자가 속으면 loss가 줄어든다.



3.7 Transposed Convolution — 왜 "역" 컨볼루션인가?

DCGAN에서 생성자가 사용하는 Transposed Convolution이 직관적이지 않을 수 있다.

일반 Conv: 큰 이미지 → 작은 feature map (다운샘플링) **Transposed Conv**: 작은 feature map → 큰 이미지 (업샘플링)

작동 원리: 입력의 각 픽셀을 "씨앗"으로 삼아, 그 주변에 필터 크기만큼의 값을 "뿌린다". 겹치는 영역은 합산된다.

이것이 "역"인 이유: 일반 Conv의 가중치 행렬을 전치(transpose)한 것으로 표현 가능하기 때문이다. 엄밀히 말하면 Conv의 역함수가 아니라, Conv의 기울기 연산에 해당한다.

3.8 Batch Normalization — 왜 학습을 안정화하는가?

DCGAN에서 BN이 핵심 기법으로 등장한다.

문제: 깊은 네트워크에서 각 레이어의 입력 분포가 학습 중에 계속 변한다. 앞 레이어의 가중치가 바뀌면 뒷 레이어의 입력 분포도 바뀐다. 이를 **Internal Covariate Shift**라고 한다.

BN의 역할: 각 레이어의 입력을 정규화하여 평균 0, 분산 1로 만든 뒤, 학습 가능한 γ, β로 스케일/이동한다.

$$\hat{x} = \frac{x - \mu_B}{\sigma_B}, \quad y = \gamma \hat{x} + \beta$$

비유: 각 레이어에 "통역관"을 배치하는 것. 앞 레이어가 어떤 언어(분포)로 말하든, 통역관이 표준어(정규 분포)로 번역해서 뒷 레이어에 전달한다.

3.9 FID — 왜 이 특정 거리를 쓰는가?

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

복잡해 보이지만, 핵심은 간단하다:

1. 이미지를 Inception V3에 통과시켜 **고수준 피처 벡터**를 추출
2. 실제 이미지들과 생성 이미지들의 피처를 각각 **가우시안으로 근사**
3. 두 가우시안 사이의 **Wasserstein-2 거리**를 계산

왜 피처 공간에서 비교하는가? 픽셀 공간에서 비교하면, 1px만 이동한 같은 이미지도 "다르다"고 판단한다. 피처 공간에서는 "의미적으로 비슷한" 이미지끼리 가깝다.

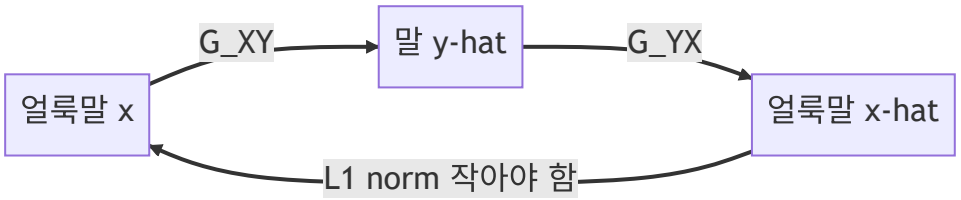
Lecture 4: CycleGAN, StyleGAN, VQ-GAN

4.1 Cycle Consistency — 왜 "왕복"이 구도를 보존하는가?

이것은 CycleGAN의 가장 핵심적인 통찰이다.

정보 이론적 설명: $G_{Y\to X}(G_{X\to Y}(x)) = x$가 성립하려면, 중간 결과 $G_{X\to Y}(x)$에 x를 **완벽하게 복원할 수 있는 정보**가 담겨 있어야 한다.

만약 $G_{X\to Y}$가 모든 얼룩말을 같은 포즈의 말로 변환한다면? → $G_{Y\to X}$가 그 말을 원래 얼룩말로 되돌릴 수 없다 (어떤 포즈였는지 정보가 사라졌으므로). → Cycle Consistency Loss가 커진다. → 따라서 구도 정보를 보존하도록 학습된다.



4.2 PatchGAN — 왜 이미지 전체가 아닌 패치 단위로 판별하는가?

핵심 통찰: 이미지 변환에서 가장 중요한 것은 **국소적 질감의 사실성**이다. 말 → 얼룩말 변환에서 "전반적으로 얼룩말처럼 보이는가?"보다 "이 부분의 줄무늬 패턴이 진짜 얼룩말의 줄무늬와 같은가?"가 더 중요하다.

전체 이미지를 보는 판별자는 "대충 얼룩말 같네" 정도의 피드백만 주지만, 70×70 패치를 보는 판별자는 "이 패치의 줄무늬 간격이 부자연스럽다"는 세밀한 피드백을 준다.

4.3 Instance Normalization vs Batch Normalization — 왜 스타일 변환에서 다른가?

CycleGAN과 StyleGAN에서 Instance Normalization이 등장한다.

	Batch Norm	**Instance Norm**
정규화 범위	배치 내 같은 채널의 모든 픽셀	**한 이미지** 내 같은 채널의 모든 픽셀
효과	학습 안정화	**스타일 정보 제거**

왜 IN이 스타일에 효과적인가? 피처 맵의 평균과 분산은 이미지의 "스타일"(콘트라스트, 밝기, 색조)을 인코딩한다. IN은 이를 정규화하여 스타일 정보를 제거하고, 이후 새로운 스타일 정보(AdaIN의 y_s, y_b)를 주입할 수 있게 만든다.

4.4 AdaIN — 왜 평균/분산 조작이 "스타일 주입"인가?

$$\mathrm{AdaIN}(x, y) = y_s \cdot \frac{x - \mu(x)}{\sigma(x)} + y_b$$

핵심 관찰 (Gatys et al., 2016에서 유래): CNN 피처 맵의 통계량(Gram matrix, 즉 채널 간 상관관계)이 이미지의 스타일을 인코딩한다. 이를 단순화하면, **평균과 분산만으로도 스타일의 많은 부분을 포착**할 수 있다.

- 정규화: 기존 스타일을 "백지" 상태로 만듦
- y_s (분산 조작): 피처의 **활성화 강도**를 조절 → 콘트라스트, 질감 강도
- y_b (평균 조작): 피처의 **기준점**을 이동 → 밝기, 색조

비유: 노래를 리믹스할 때, 원곡의 볼륨과 피치를 초기화한 뒤(정규화), 새로운 볼륨(y_s)과 피치(y_b)를 적용하는 것과 같다.

4.5 VQ-GAN — 왜 VQ-VAE에 GAN Loss를 추가하는가?

VQ-VAE의 복원 이미지는 L2 loss로 학습되므로 **흐릿하다**. L2 loss는 "평균적으로 맞는" 방향으로 학습하기 때문이다.

GAN loss(판별자)를 추가하면, 복원 이미지가 "진짜처럼 보이는가?"도 평가된다. 이것은 **선명도와 디테일을 강제**한다.

$$\mathcal{L}_{\text{VQ-GAN}} = \underbrace{\|x - \hat{x}\|^2}_{\text{L2: 구조 보존}} + \underbrace{\mathcal{L}_{\text{VQ}}}_{\text{코드북 학습}} + \underbrace{\lambda \cdot \mathcal{L}_{\text{GAN}}}_{\text{선명도 강제}}$$

4.6 Perceptual Loss — 왜 픽셀 차이가 아닌 피처 차이를 재는가?

VQ-GAN에서 perceptual loss가 사용된다.

픽셀 L2의 문제: 1px 이동한 같은 이미지도 L2가 크다. 사람 눈에는 동일한데!

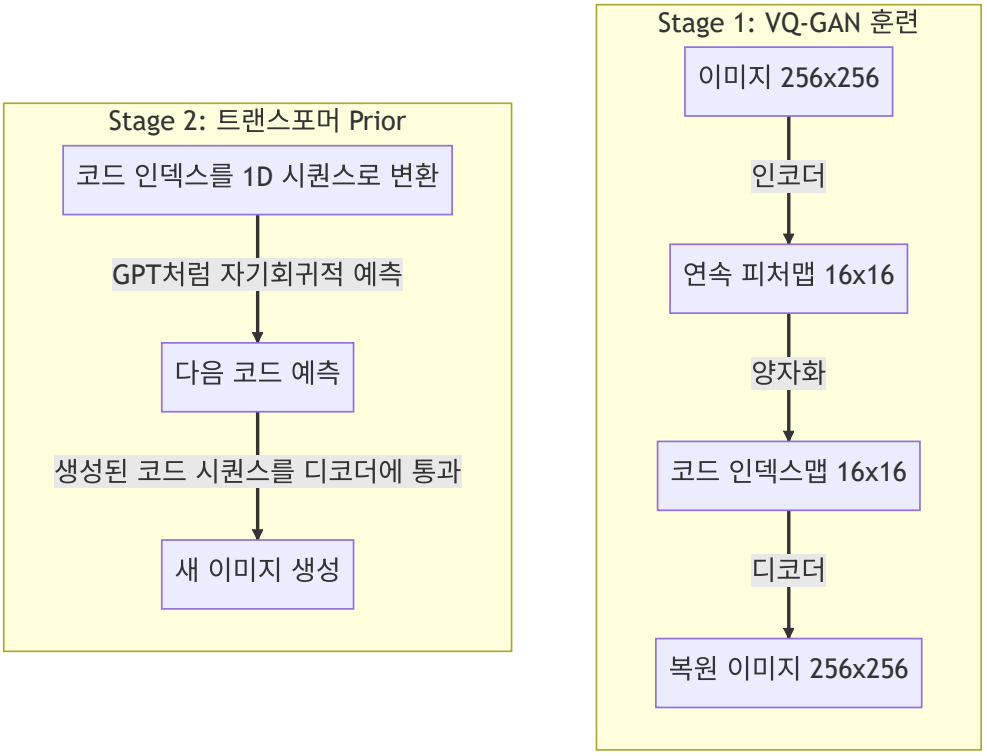
Perceptual Loss: 사전 학습된 VGG 같은 네트워크의 중간 피처를 비교.

$$\mathcal{L}_{\text{perceptual}} = \sum_{l} \|\phi_l(x) - \phi_l(\hat{x})\|^2$$

여기서 ϕ_l은 VGG의 l번째 레이어 피처. 이 피처 공간에서는 "의미적으로 비슷한" 이미지끼리 가깝고, 단순한 기하학적 변환에 강건하다.

4.7 트랜스포머로 이미지를 생성한다? — VQ-GAN Stage 2의 핵심

VQ-GAN의 2단계가 매우 중요하다:



핵심 통찰: 이미지를 "코드의 시퀀스"로 변환하면, **텍스트 생성과 동일한 문제**가 된다. GPT가 다음 단어를 예측하듯, 트랜스포머가 다음 코드를 예측한다. 이것이 DALL-E의 기본 원리다.

Lecture 5: DALL-E & Score-Based Models

5.1 Score Function — 왜 확률 대신 "기울기"를 학습하는가?

Score-based 모델의 핵심 아이디어:

$$s_\theta(x) = \nabla_x \log p(x)$$

"확률 분포의 로그에 대한 기울기"를 학습한다. 왜?

확률을 직접 학습하는 것의 문제: $p(x)$는 정규화 상수 $Z = \int p(x)dx$를 계산해야 한다. 고차원에서 이 적분은 intractable하다.

Score의 장점: $\nabla_x \log p(x) = \nabla_x \log \frac{\tilde{p}(x)}{Z} = \nabla_x \log \tilde{p}(x)$ — 정규화 상수 Z가 사라진다! (상수의 기울기는 0이므로)

직관적 의미: Score는 "데이터 분포의 가장 밀도가 높은 방향을 가리키는 화살표"다.

이것은 **등산**과 같다. 현재 위치에서 가장 가파른 오르막 방향(score)으로 한 걸음 이동하면, 결국 산꼭대기(데이터 분포의 모드)에 도달한다.

5.2 Langevin Dynamics — 왜 노이즈를 추가하면서 샘플링하는가?

$$x_{t+1} = x_t + \frac{\epsilon}{2}\nabla_x \log p(x_t) + \sqrt{\epsilon} \cdot z_t, \quad z_t \sim \mathcal{N}(0, I)$$

score 방향으로 이동하면서 **노이즈까지 추가**한다. 왜?

노이즈 없이 이동하면: gradient ascent가 되어 가장 가까운 모드(산봉우리)에 갇힌다. 다른 모드를 탐험하지 못한다.

노이즈를 추가하면: 확률적으로 탐험하여 여러 모드를 방문한다. 충분히 오래 실행하면 $p(x)$에서의 샘플이 된다 (MCMC 이론).

비유: 등산에서 무조건 오르막으로만 가면 가장 가까운 봉우리에 갇긴다. 가끔 랜덤하게 옆으로 가면 더 높은 봉우리를 발견할 수 있다.

5.3 Score Matching — 왜 이 특정 목적 함수가 작동하는가?

$$\mathcal{L} = \mathbb{E}_{p(x)}\left[\|s_\theta(x) - \nabla_x \log p(x)\|^2\right]$$

문제: $\nabla_x \log p(x)$를 모르는데 어떻게 학습하나?

Denoising Score Matching의 트릭: 깨끗한 데이터 x에 노이즈를 추가한 $\tilde{x} = x + \sigma z$를 만들면, $\nabla_{\tilde{x}} \log p(\tilde{x}|x) = -\frac{\tilde{x}-x}{\sigma^2} = -\frac{z}{\sigma}$ 이것은 알려진 값이다!

따라서 학습 목표:

$$\mathcal{L} = \mathbb{E}\left[\|s_\theta(\tilde{x}) - (-z/\sigma)\|^2\right]$$

직관: "노이즈가 추가된 이미지에서, 원래 깨끗한 이미지 방향을 예측하라." 이것이 후에 DDPM의 "노이즈 예측"과 연결된다.

Lecture 6: 보충 강의

6.1 Self-Attention — 왜 CNN만으로는 부족한가?

CNN은 **지역적(local)** 패턴만 감지한다. 3×3 필터는 주변 9개 픽셀만 본다. 이미지의 먼 부분 사이의 관계(예: 왼쪽 눈과 오른쪽 눈의 대칭)를 포착하려면 매우 깊은 네트워크가 필요하다.

Self-Attention은 이미지의 **모든 위치 쌍** 사이의 관계를 한 번에 계산한다:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q: "내가 뭘 찾고 있는가?" (Query)
- K: "나는 이런 정보를 갖고 있다" (Key)
- V: "내 실제 정보" (Value)
- QK^T: 모든 위치 쌍의 유사도 계산
- softmax: 유사도를 확률로 변환
- $\times V$: 유사한 위치의 정보를 가중 합산

$\sqrt{d_k}$로 나누는 이유: Q와 K의 내적 값이 d_k가 커지면 절대값이 커진다 (각 원소가 독립이면 분산이 d_k에 비례). 값이 크면 softmax가 하나의 위치에 극단적으로 집중(saturate)된다. $\sqrt{d_k}$로 나눠 적절한 범위로 조절한다.

6.2 Cross-Attention — 조건부 생성에서 어떻게 작동하는가?

텍스트 조건부 이미지 생성에서:

- Q: 이미지 피처 (from 디코더)
- K, V: 텍스트 임베딩 (from 인코더)

이미지의 각 위치가 "이 부분을 생성하는 데 텍스트의 어떤 단어가 관련 있는가?"를 물어보는 것이다.

5.4 dVAE vs VQ-VAE — 왜 DALL-E는 다른 양자화를 쓰는가?

VQ-VAE는 `argmin`으로 가장 가까운 코드북 벡터를 **딱 하나** 고른다. 이것은 이산적 선택이라 미분이 불가능하고, Straight-Through Estimator(STE)라는 근사적 방법으로 역전파한다.

DALL-E에 사용된 dVAE(discrete VAE)는 **Gumbel-Softmax**라는 다른 전략을 택한다.

두 방식의 핵심 차이:

	VQ-VAE	dVAE (DALL-E)
양자화 방식	$z_q = \text{codebook}[\text{argmin}_k \lvert z_e - e_k \rvert]$	$z_q = \sum_k \text{softmax}(\text{logits}/\tau)_k \cdot e_k$
선택 방식	Hard (하나만 선택)	Soft (가중 평균)
미분 가능?	불가 (STE로 우회)	**가능** (temperature를 통해)
학습 안정성	codebook collapse 위험	더 안정적

Gumbel-Softmax가 작동하는 원리:

범주형 분포에서 샘플링하는 것은 미분 불가능하다 ("3번 코드를 선택"이라는 이산적 결정). Gumbel-Softmax 트릭은 이를 연속적 근사로 바꾼다:

$$y_k = \frac{\exp((\log \pi_k + g_k)/\tau)}{\sum_j \exp((\log \pi_j + g_j)/\tau)}$$

여기서 $g_k \sim \text{Gumbel}(0, 1)$은 Gumbel 노이즈, τ는 온도(temperature) 파라미터다.

온도 τ의 역할:

- $\tau \to \infty$: 모든 코드에 균일한 가중치 → 탐험(exploration)
- $\tau \to 0$: one-hot에 가까워짐 → 착취(exploitation), VQ-VAE의 argmin과 동일
- **Annealing**: 학습 초기에는 높은 τ로 다양한 코드를 탐색하고, 점차 낮춰서 선명한 선택으로 수렴

```python
import torch
import torch.nn.functional as F

def gumbel_softmax(logits, tau=1.0, hard=False):
    """Gumbel-Softmax 양자화 (미분 가능한 이산 선택)"""
    # Gumbel 노이즈 생성: -log(-log(U)), U ~ Uniform(0,1)
    gumbels = -torch.log(-torch.log(torch.rand_like(logits) + 1e-20

    # 노이즈를 logits에 더하고 temperature로 스케일링
    y_soft = F.softmax((logits + gumbels) / tau, dim=-1)

    if hard:
        # 순전파: one-hot (이산), 역전파: soft (연속) — STE와 유사한 
        index = y_soft.argmax(dim=-1, keepdim=True)
        y_hard = torch.zeros_like(logits).scatter_(-1, index, 1.0)
        return y_hard - y_soft.detach() + y_soft  # 역전파 시 y_soft
    return y_soft

# 사용 예시
logits = torch.randn(1, 8192)  # 8192개 코드북에 대한 logit
tau = 1.0  # 온도 파라미터 (학습 중 점점 낮춤)

# 부드러운 선택 → 코드북 벡터의 가중 평균
soft_selection = gumbel_softmax(logits, tau=tau)  # [1, 8192] 확률 
# z_q = soft_selection @ codebook  # 가중 평균으로 양자화된 벡터 생성
```

비유: VQ-VAE는 식당 메뉴에서 하나를 **딱 골라야** 하는 것이고, dVAE는 뷔페에서 여러 요리를 **비율을 조절해 담을 수 있는** 것이다. 뷔페 방식이 더 유연하고, "어떤 조합이 좋았는지"를 학습하기도 쉽다.

5.5 Teacher Forcing — 왜 학습과 생성이 다른가?

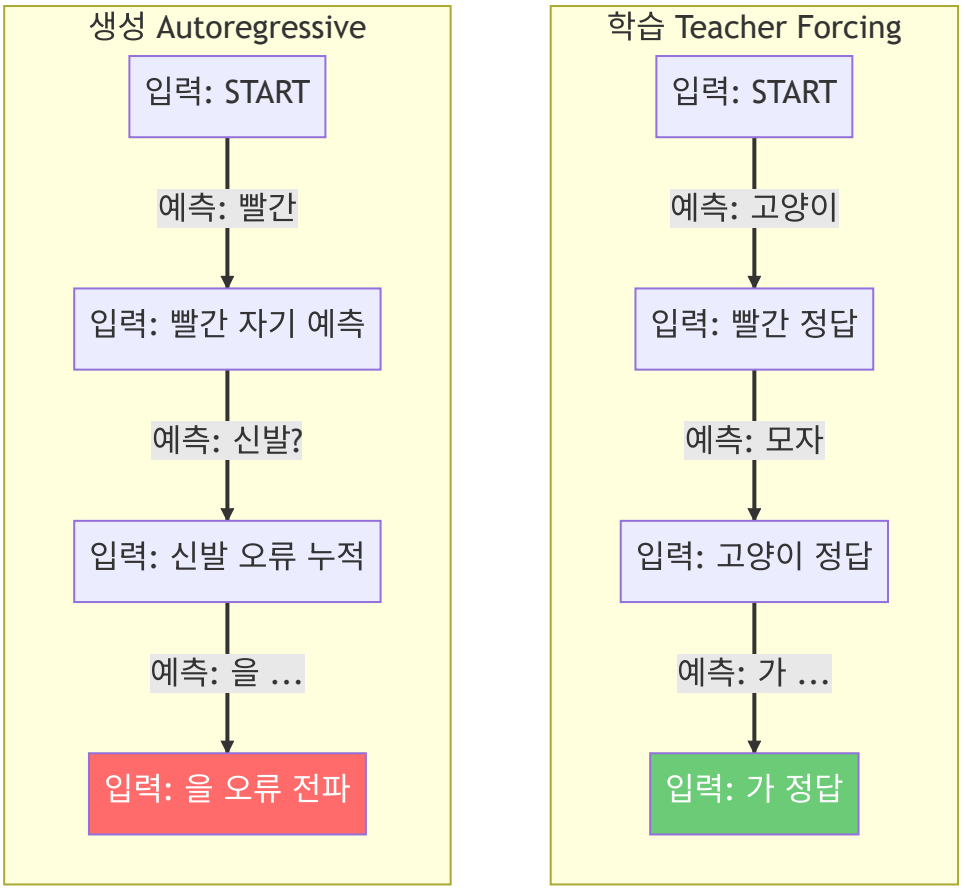
자기회귀 모델(GPT, PixelRNN, DALL-E의 트랜스포머)에서 학습할 때와 생성할 때의 동작이 근본적으로 다르다.

학습 시 (Teacher Forcing):

- 정답 시퀀스 $[x_1, x_2, x_3, \ldots]$를 미리 알고 있다
- 각 시점에서 **정답 토큰**을 다음 입력으로 사용
- 설사 $t = 2$에서 모델이 틀린 예측을 했더라도, $t = 3$의 입력은 정답 x_2

생성 시 (Autoregressive Decoding):

- 정답이 없다
- 각 시점에서 **모델 자신의 예측**을 다음 입력으로 사용
- $t = 2$에서 틀리면, 그 오류가 $t = 3$, $t = 4$, ... 로 **누적**된다



Exposure Bias (노출 편향) 문제:

학습 시에는 항상 "올바른 맥락"만 보고 예측을 배운다. 하지만 생성 시에는 자기가 만든 "틀릴 수 있는 맥락"을 보고 예측해야 한다. 모델이 한 번도 경험하지 못한 상황(자기 오류)에 **노출(expose)**되는 것이다.

비유: Teacher Forcing은 **모범 답안을 보면서 다음 줄을 쓰는 연습**이고, 실제 생성은 **아무 참고 없이 백지에서 작문하는 것**이다. 연습과 실전의 괴리가 클수록 성능 차이가 난다.

완화 방법들:

- **Scheduled Sampling**: 학습 중 일정 확률로 자기 예측을 입력으로 사용 (점진적으로 비율 증가)
- **Beam Search**: 생성 시 여러 후보를 동시에 유지하여 오류 누적 위험 분산
- **Diffusion 모델**: 자기회귀가 아니므로 이 문제 자체가 없음 (DALL-E 2가 Diffusion으로 전환한 이유 중 하나)

5.6 Sliced Score Matching — 왜 "슬라이스"하는가?

Score Matching의 원래 목적 함수를 풀어쓰면:

$$\mathcal{L}_{\text{SM}} = \mathbb{E}_{p(x)}\left[\text{tr}(\nabla_x s_\theta(x)) + \frac{1}{2}\|s_\theta(x)\|^2\right]$$

여기서 $\text{tr}(\nabla_x s_\theta(x))$는 score 함수의 **자코비안의 대각합(trace)**이다.

문제: x가 n차원이면, 자코비안은 $n \times n$ 행렬이다. 이것을 계산하려면 역전파를 n번 해야 하므로, $O(n^2)$의 계산 비용이 든다. 고해상도 이미지에서 n이 수만~수십만이면 사실상 불가능하다.

Sliced Score Matching (SSM)의 아이디어:

n차원 전체의 자코비안 대신, **랜덤한 방향 v로 사영(projection)**하여 스칼라 값을 계산한다:

$$\mathcal{L}_{\text{SSM}} = \mathbb{E}_{p(v)}\mathbb{E}_{p(x)}\left[v^T \nabla_x s_\theta(x) v + \frac{1}{2}(v^T s_\theta(x))^2\right]$$

여기서 v는 단위 구 위의 랜덤 벡터이고, $v^T \nabla_x s_\theta(x) v$는 스칼라이므로 역전파 1번으로 계산 가능하다.

비유: 3D 물체의 정확한 형태를 알려면 3D 스캐너가 필요하다 ($O(n^2)$). 하지만 랜덤한 각도에서 **그림자(projection)**를 여러 번 찍으면 ($O(n)$), 충분히 많은 그림자로부터 원래 형태를 추정할 수 있다. 이것이 "슬라이스"의 직관이다.

Denoising Score Matching (DSM) — 더 실용적인 대안:

실전에서는 SSM보다 Denoising Score Matching이 더 많이 사용된다:

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{x\sim p(x)}\mathbb{E}_{\tilde{x}\sim q(\tilde{x}|x)}\left[\|s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q(\tilde{x}|x)\|^2\right]$$

가우시안 노이즈를 사용하면 $q(\tilde{x}|x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$이므로:

$$\nabla_{\tilde{x}} \log q(\tilde{x}|x) = -\frac{\tilde{x} - x}{\sigma^2}$$

이것은 **알려진 값**이다! 자코비안 계산이 전혀 필요 없고, 단순히 "노이즈 방향을 예측"하는 문제가 된다. 이것이 DDPM에서 ϵ-prediction으로 직접 이어진다.

6.3 Energy-Based Models (EBM) — Score의 기원

Score-based 모델이 왜 등장했는지를 이해하려면, 그 뿌리인 **에너지 기반 모델(EBM)**부터 살펴야 한다.

에너지 함수의 직관:

물리학에서 에너지가 낮은 상태가 안정적이듯, 에너지 기반 모델에서는 $E_\theta(x)$가 낮은 데이터가 "그럴듯한" 데이터다.

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z(\theta)}, \quad Z(\theta) = \int \exp(-E_\theta(x))dx$$

Z(θ)의 문제 — 정규화 상수의 난관:

$Z(\theta)$는 모든 가능한 x에 대해 $\exp(-E_\theta(x))$를 적분해야 한다. 이미지처럼 고차원인 경우, 이 적분은 계산 불가능(intractable)하다.

$Z(\theta)$를 모르면:

- $p_\theta(x)$의 정확한 값을 계산할 수 없다
- Maximum Likelihood 학습이 직접적으로 불가능하다

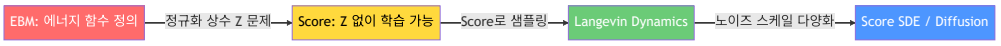
Score가 Z를 제거하는 마법:

$$\nabla_x \log p_\theta(x) = \nabla_x \left[\log \exp(-E_\theta(x)) - \log Z(\theta)\right] = -\nabla_x E_\theta(x) - 0$$

$\log Z(\theta)$는 x에 대한 상수이므로, x에 대해 미분하면 사라진다. 따라서:

$$\text{Score} = s_\theta(x) = -\nabla_x E_\theta(x)$$

정규화 상수를 전혀 알 필요가 없다!



논리적 흐름: EBM은 유연한 확률 모델이지만 Z 때문에 학습이 어렵다 → Score를 사용하면 Z 없이 학습할 수 있다 → Score를 알면 Langevin Dynamics로 샘플링할 수 있다 → 여러 노이즈 스케일의 Score를 학습하면 Diffusion 모델이 된다. 이것이 **EBM → Score → Diffusion**이라는 지적 계보다.

6.4 생성 모델 평가 지표 — FID, IS, LPIPS의 직관

생성 모델의 "좋은 이미지"란 무엇인가? 이를 정량화하는 주요 지표들의 직관을 정리한다.

Inception Score (IS) — "선명하면서도 다양한가?"

$$\text{IS} = \exp\left(\mathbb{E}_x\left[D_{KL}(p(y|x)\|p(y))\right]\right)$$

여기서 $p(y|x)$는 Inception 네트워크가 생성 이미지 x에 대해 예측한 클래스 분포이고, $p(y) = \mathbb{E}_x[p(y|x)]$는 모든 생성 이미지에 대한 평균 클래스 분포다.

직관적으로 두 가지를 동시에 측정한다:

- **품질(Quality)**: 각 이미지의 $p(y|x)$가 뾰족해야 함 → "이 이미지는 확실히 고양이다" (모호하지 않음)

- **다양성(Diversity)**: 전체 $p(y)$가 균일해야 함 → "고양이, 개, 자동차 등 다양한 것을 생성"

IS의 한계: 실제 데이터와 비교하지 않는다. 극단적으로, ImageNet의 1000개 클래스를 각각 1장씩 완벽하게 외운 모델도 높은 IS를 받는다.

FID (Frechet Inception Distance) — "실제와 얼마나 비슷한가?"

$$\mathrm{FID} = \|\mu_r - \mu_g\|^2 + \mathrm{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Lecture 3에서 다뤘지만, 추가 직관을 더하면:

FID는 두 가우시안 사이의 거리인데, **왜 가우시안으로 근사해도 되는가?** Inception 피처 공간에서 자연 이미지들의 분포는 경험적으로 가우시안에 가깝다는 관찰에 기반한다. 완벽하지는 않지만, 실용적으로 잘 동작한다.

LPIPS (Learned Perceptual Image Patch Similarity) — "사람 눈에 비슷한가?"

$$\mathrm{LPIPS}(x, \hat{x}) = \sum_l \|w_l \odot (\phi_l(x) - \phi_l(\hat{x}))\|^2$$

여기서 ϕ_l은 사전학습된 네트워크(AlexNet, VGG 등)의 l번째 레이어 피처, w_l은 학습된 가중치이다.

왜 픽셀 L2보다 나은가? 사람의 시각 체계는 저수준 픽셀 차이가 아니라, 고수준 패턴(엣지, 텍스처, 구조)의 유사성을 기반으로 판단한다. 1px 이동은 L_2 거리가 크지만 사람 눈에는 동일하다. LPIPS는 이런 **지각적 유사성**을 반영한다.

지표 비교 요약:

지표	측정 대상	실제 데이터 필요?	단일 이미지 평가?	주요 용도
IS	품질 + 다양성	아니오	아니오 (집합)	생성 품질 빠른 확인
FID	실제와의 분포 거리	**예**	아니오 (집합)	모델 간 비교 (가장 보편적)
LPIPS	두 이미지의 지각적 거리	해당 없음 (쌍)	**예** (이미지 쌍)	복원/변환 품질 평가

6.5 SDE/ODE 통합 프레임워크 — Song의 대통합

Yang Song et al. (ICLR 2021, "Score-Based Generative Modeling through Stochastic Differential Equations")는 DDPM과 Score-based 모델이 **같은 프레임워크의 특수한 경우**임을 보였다.

Forward SDE — 노이즈를 점진적으로 추가:

$$dx = f(x,t)dt + g(t)dW$$

- $f(x,t)$: 드리프트(drift) — 결정적 변화
- $g(t)$: 확산(diffusion) 계수 — 노이즈 강도
- dW: 위너 과정 (브라운 운동의 미소 증분)

시간이 흐르면($t : 0 \to T$) 데이터가 점점 순수 노이즈로 변한다.

Reverse SDE — 노이즈에서 데이터로 복원:

$$dx = \left[f(x,t) - g(t)^2 \nabla_x \log p_t(x)\right] dt + g(t) d\bar{W}$$

핵심: **score** $\nabla_x \log p_t(x)$만 알면 역방향 과정을 시뮬레이션할 수 있다!

Probability Flow ODE — 확률적 요소를 제거한 버전:

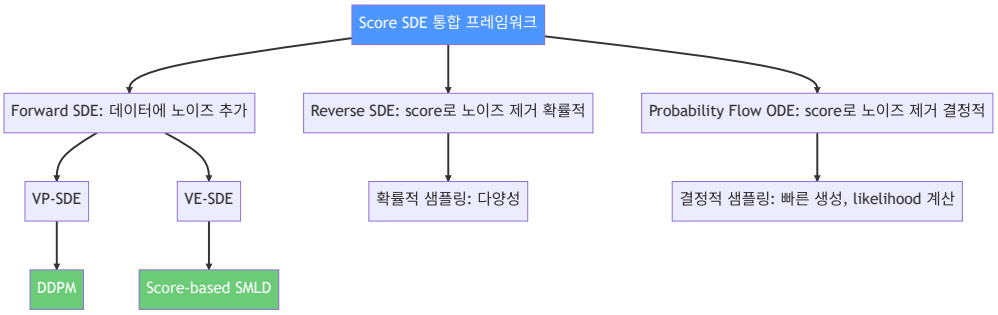
$$dx = \left[f(x,t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x)\right] dt$$

놀라운 사실: 이 ODE는 Reverse SDE와 **동일한 주변 분포** $p_t(x)$를 공유한다. 즉, 각 시점 t에서의 확률 분포가 같다. 하지만 ODE는 결정적이므로:

- 동일한 시작점에서 항상 같은 결과
- likelihood를 정확히 계산 가능 (연속 normalizing flow)
- 더 적은 스텝으로 빠르게 생성 가능

DDPM과 Score-based 모델의 통합:

모델	SDE 유형	$f(x,t)$	$g(t)$
DDPM	VP-SDE (Variance Preserving)	$-\frac{1}{2}\beta(t)x$	$\sqrt{\beta(t)}$
Score-based (SMLD)	VE-SDE (Variance Exploding)	0	$\sqrt{\frac{d[\sigma^2(t)]}{dt}}$



핵심 통찰: DDPM 저자들은 "노이즈를 점진적으로 추가/제거"라는 관점에서, Score-based 저자들은 "score를 학습하여 Langevin dynamics로 샘플링"이라는 관점에서 접근했지만, **둘 다 같은 수학적 구조(SDE)의 서로 다른 파라미터화**였다. Song의 논문이 이 대통합을 이루어낸 것이다.

6.6 Negative Prompt — 왜 "원하지 않는 것"을 명시하면 효과가 있는가?

Stable Diffusion 등에서 사용하는 Negative Prompt는 Classifier-Free Guidance(CFG)의 확장이다.

표준 CFG 복습:

$$\tilde{\epsilon} = \epsilon_{\text{uncond}} + w \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

여기서:

- ϵ_{uncond}: 조건 없이 예측한 노이즈 (기준점)
- ϵ_{cond}: 텍스트 조건으로 예측한 노이즈 (목표 방향)
- w: guidance scale (방향의 강도)

기하학적으로: "무조건적 방향에서 조건적 방향으로 w배만큼 외삽(extrapolate)한다."

Negative Prompt의 수정:

$$\tilde{\epsilon} = \epsilon_{\text{neg}} + w \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{neg}})$$

단순히 ϵ_{uncond}를 ϵ_{neg}(부정 프롬프트로 예측한 노이즈)로 **교체**한 것이다!

왜 효과가 있는가?

표준 CFG에서 "기준점"은 "아무 조건도 없는 이미지"였다. Negative Prompt를 쓰면 기준점이 "원하지 않는 속성의 이미지"가 된다. 외삽 방향이 바뀌므로:

- 표준 CFG: "아무것도 아닌 것" → "원하는 것" 방향으로 이동
- Negative Prompt: **"원하지 않는 것"** → "원하는 것" 방향으로 이동 → 원하지 않는 속성에서 **더 강하게 멀어짐**

실전 예시:

Positive Prompt	Negative Prompt	효과
"portrait photo of a woman"	"low quality, blurry, deformed"	저품질 특성에서 멀어짐 → 고품질

"landscape painting"	"photorealistic, photograph"	사진 같은 특성에서 멀어짐 → 더 회화적
"cute cat"	"scary, dark, horror"	공포 분위기에서 멀어짐 → 더 밝고 귀여운 결과

수학적 직관: 벡터 공간에서 $\vec{v}_{\text{result}} = \vec{v}_{\text{neg}} + w(\vec{v}_{\text{pos}} - \vec{v}_{\text{neg}})$는 $\vec{v}_{\text{neg}}$에서 $\vec{v}_{\text{pos}}$ 방향으로의 선형 외삽이다. $w > 1$이면 $\vec{v}_{\text{pos}}$를 넘어서까지 가므로, 원하는 속성이 과장되고 원하지 않는 속성은 더 강하게 억제된다.

다음 파일(보충_Lecture07_12_직관적이해.md)에서 Lecture 7~12를 이어서 다룹니다.